Declan Sheehan

Cosc 320-001

Dr. Anderson

10 September, 2019

README

Theoretic Time Complexity:

Sort	Best	Worst
Merge Sort	O(nlog(n))	O(nlog(n))
n=100	200	200
n=1000	3000	3000
n=10000	40000	40000
Quick Sort	O(nlog(n))	O(n^2)
n=100	200	10000
n=1000	3000	1000000
n=10000	40000	100000000

Absolute Timing Scale:

The timing scale for the runtime matches well with the time complexities for merge and quick sort. From the data it is hard to say it entirely follows the time complexity, however in the data we can see that quick sort takes a fair bit longer to sort 1M than merge sort (with roughly a factor of 81). With that being said, the data that came from the runtime replicates the theoretic time complexities, because quick sort performs between O(n^2) and O(nlog(n)) respectively.

Algorithm Analysis:

Each respective algorithm performed as expected, as the merge sort outperformed quick sort by a fair factor. For the most part, each random sort ran longer than a presorted array, except for the quick sorts data. It is visible that quick sorts presorted arrays take longer as the size of the array increases. In most cases, the presorted arrays take less time than the reversed arrays which is as expected. Lastly, for duplicated arrays, it was a toss-up between the two; some would take longer and some would take shorter.

Sort Comparison:
Comparing merge and quick sort with bubble sort, there is a tremendous difference between the two. For instance, the 1M time elapsed for bubble sort was beat by merge sort with a factor of over **10,000.** To add to it, quick sort had a factor of: ~**500.** The bubble sort algorithm swapped well over 1 billion times, while the merge and quick sort algorithms swap a lot less.

Code Improvements:
As far as I am concerned, the only improvements that can be made is to either run a lot more array sizes, and to make the output even more readable.